

Experiment No. 5

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

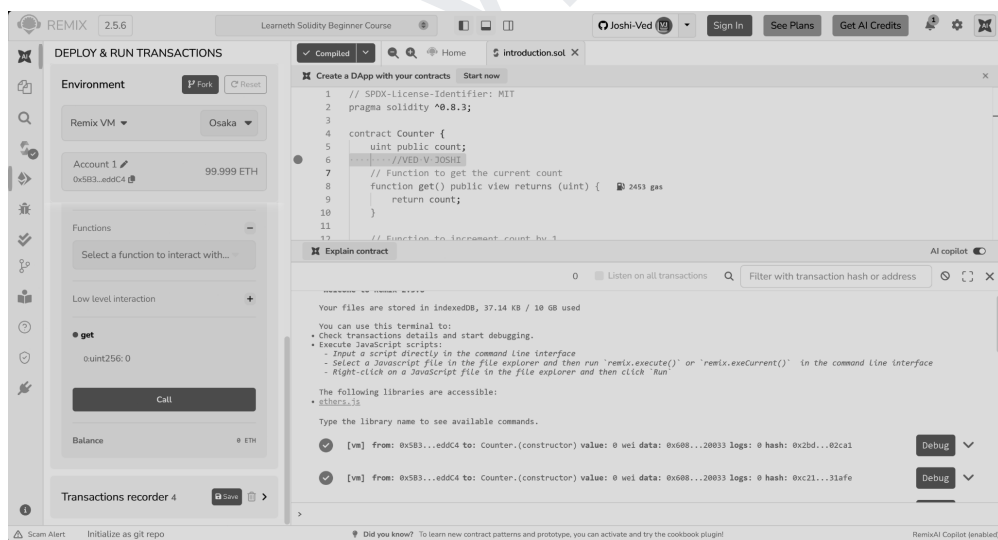
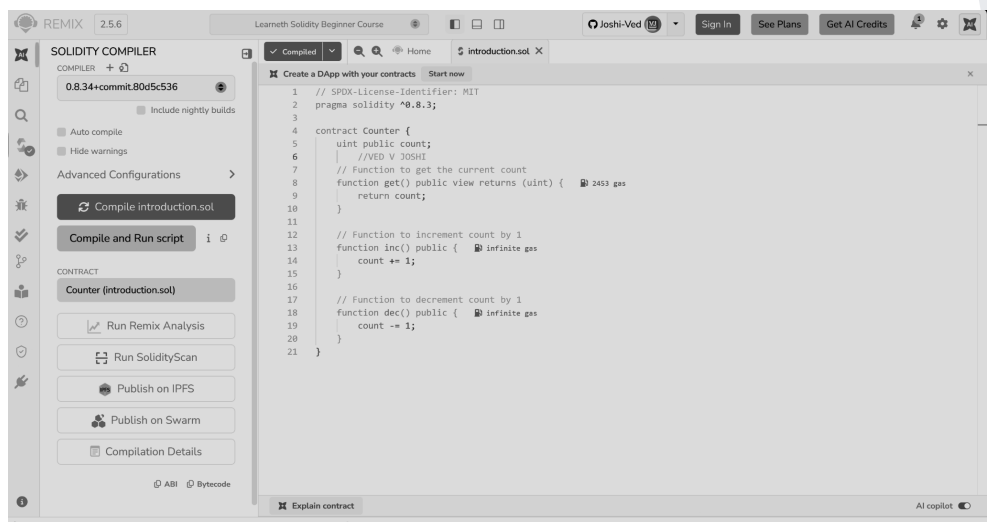
6. Chain Replacement

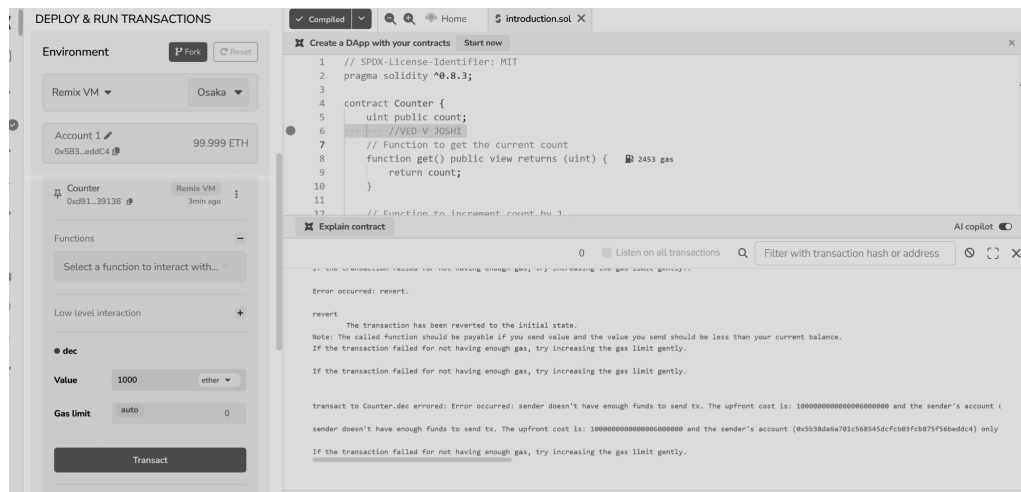
When `/replace_chain` is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

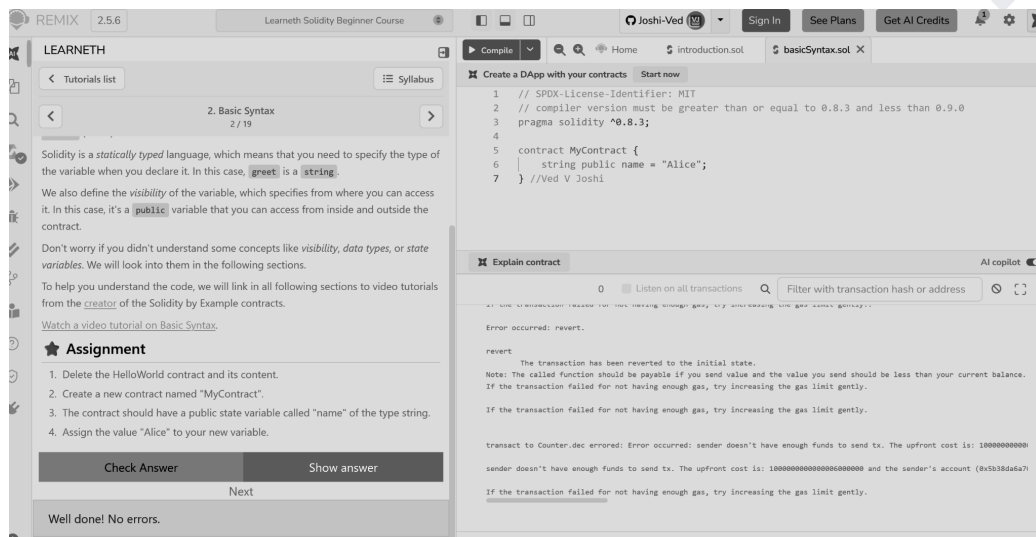
Outputs:

Tutorial 1

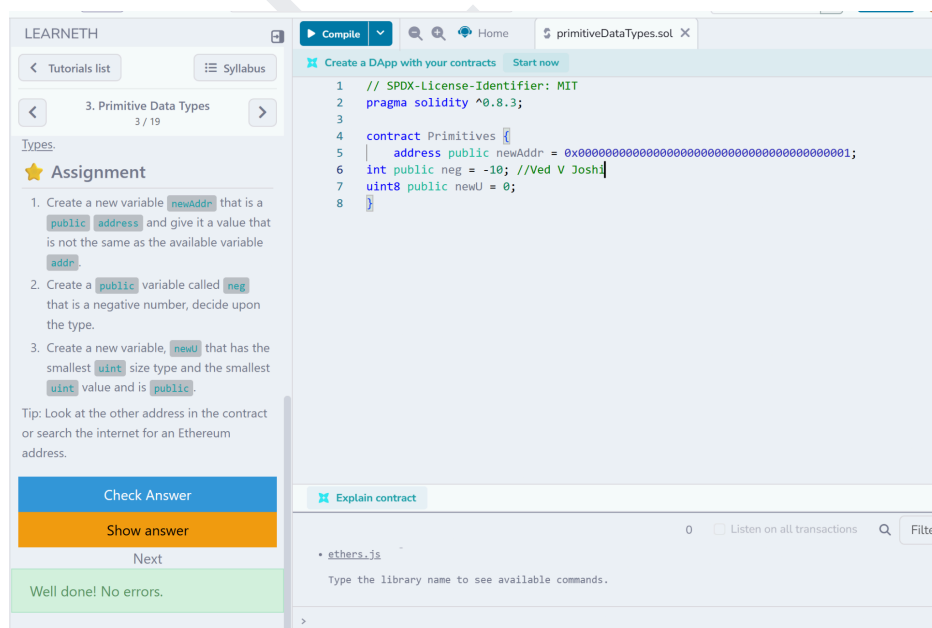




Tut-2



Tut-3



Tut 4

REMIX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

4. Variables 4 / 19

A list of all Global Variables is available in the Solidity documentation.

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public {
11        // Local variables are not saved to the blockchain.
12        uint i = 456;
13
14        // Global variables
15        uint timestamp = block.timestamp; // Current block timestamp
16        address sender = msg.sender;
17
18        blockNumber = block.number; // Ved V Joshi
19    }
20 }
```

Explain contract

0 Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 5

REMIX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable 5 / 19

state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Create a DApp with your contracts Start now

```
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // Assignment 1: Boolean state variable
9     bool public b = true;
10
11    // Write to the state variable
12    function set(uint _num) public {
13        num = _num;
14    }
15
16    // Read the number Ved V Joshi
17    function get() public view returns (uint) {
18        return num;
19    }
20
21    // Assignment 2: Read the Boolean variable
22    function get_b() public view returns (bool) {
23        return b;
24    }
25 }
```

Explain contract

0 Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 6

REMUX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.2 Functions - View and Pure 6 / 19

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addTnx2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addTnx(uint y) public view returns (uint) {
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) {
14        return i + j;
15    }
16    //Ved V Joshi
17
18    // Assignment: Modify x by adding y to it.
19    function addTnx2(uint y) public {
20        x = x + y;
21    }
22 }
```

Explain contract

0 Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 7

REMUX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.3 Functions - Modifiers and Constructors 7 / 19

contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

- Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
- Make sure that `x` can only be increased.
- The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

```
34 locked = true;
35 _;
36 locked = false;
37 }
38
39 function decrement(uint i) public noReentrancy {
40     x -= i;
41
42     if (i > 1) {
43         decrement(i - 1);
44     }
45 }
46
47 // Assignment modifier Ved V Joshi
48 modifier increaseOnly(uint _x) {
49     x += _x;
50     _;
51 }
52
53 // Assignment function
54 function increaseX(uint _x) public increaseOnly(_x) {
55 }
56 }
```

Explain contract

0 Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 8

https://remix.ethereum.org/?#activate=udapp,solidity,LearnEth&lang=en&optimize&runs=200&evmVersion=version=soljson-v0.8.34+commit.80d5c536.js

REMIK 2.5.6 Learneth Solidity Beginner Course Joshi-Ved Sign In See

LEARNETH

Tutorials list 5.4 Functions - Inputs and Outputs 8 / 19

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Create a DApp with your contracts Start now

```
65 // Assignment: return -2 and true without return statement.
66 function returnTwo() 472 gas
67 {
68     public
69     pure
70     returns (
71         int x,
72         bool b
73     )
74 {
75     x = -2;
76     b = true;
77 }
78
79 // Can use array for input Ved V Joshi
80 function arrayInput(uint[] memory _arr) public {} infinite gas
81
82 // Can use array for output
83 uint[] public arr;
84
85 function arrayOutput() public view returns (uint[] memory) { infinite gas
86     return arr;
87 }
88 }
```

Explain contract

0 Listen on all transactions Filter with

• ethers.js

Type the library name to see available commands.

Tut 9

REMIK 2.5.6 Learneth Solidity Beginner Course Joshi-Ved Sign In See

LEARNETH

Tutorials list 6. Visibility 9 / 19

(lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Create a DApp with your contracts Start now

```
35 string internal internalVar = "my internal variable";
36 string public publicVar = "my public variable";
37 }
38
39 contract Child is Base {
40     // Internal function can be called inside child contracts.
41     function testInternalFunc() infinite gas
42     {
43         public
44         pure
45         override
46         returns (string memory)
47     {
48         return internalFunc();
49     }
50
51 // Assignment Ved V Joshi
52 function testInternalVar() infinite gas
53 {
54     public
55     view
56     returns (
57         string memory,
58         string memory
59     )
60 }
```

Explain contract

0 Listen on all transactions Filter with

• ethers.js

Type the library name to see available commands.

Tut 10

REMIX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.1 Control Flow - If/Else 10 / 19

function returns 1

Watch a video tutorial on the If/Else statement.

★ Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

```
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint x) public pure returns (uint) {
16        // Shorthand way to write if / else statement Ved V Joshi
17        return x < 10 ? 1 : 2;
18    }
19
20    // Assignment: Check whether a number is even or odd
21    function evenCheck(uint x) public pure returns (bool) {
22        return x % 2 == 0 ? true : false;
23    }
24 }
```

Tut 11

REMIX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

7.2 Control Flow - Loops 11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

★ Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

```
4 contract Loop {
5     uint public count;
6
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10            if (i == 3) {
11                continue;
12            }
13
14            if (i == 5) {
15                break;
16            }
17
18            count++;
19        }
20
21        // Ved V Joshi
22        uint j;
23        while (j < 10) {
24            j++;
25        }
26
27        // Make count equal to 9
28        count += 5;
29    }
30 }
```

Tut 12

REMIX 2.5.6 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

8.1 Data Structures - Arrays 12 / 19

element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

- Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
- Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

```
50
51
52 function remove(uint index) public {
53     // Move the last element into the place to delete
54     arr[index] = arr[arr.length - 1];
55
56     // Remove the last element
57     arr.pop();
58 }
59
60 function test() public {
61     arr.push(1);
62     arr.push(2);
63     arr.push(3);
64     arr.push(4);
65     // [1, 2, 3, 4]
66
67     remove(1);
68     // [1, 4, 3]
69
70     remove(2);
71     // [1, 4]
72 }
73 }
```

Tut 13

LEARNETH 2.5.6 Learneth Solidity Beginner Course

8.2 Data Structures - Mappings 13 / 19

to the default value 0.

Setting values

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `set` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer **Show answer**

Next

Well done! No errors.

Compile Create a DApp with your contracts Start now

```
22
23
24 function get(
25     address _addr1,
26     uint _i
27 ) public view returns (bool) {
28     return nested[_addr1][_i];
29 }
30
31 function set(
32     address _addr1,
33     uint _i,
34     bool _boo
35 ) public {
36     nested[_addr1][_i] = _boo;
37 }
38 //Ved V Joshi
39 function remove(
40     address _addr1,
41     uint _i
42 ) public {
43     delete nested[_addr1][_i];
44 }
```

Explain contract

0 ☐ Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 14

LEARNETH 2.5.6 Learneth Solidity Beginner Course

8.3 Data Structures - Structs 14 / 19

Positional parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer **Show answer**

Next

Well done! No errors.

Compile Create a DApp with your contracts Start now

```
37 returns (string memory text, bool completed)
38 {
39     Todo storage todo = todos[_index];
40     return (todo.text, todo.completed);
41 }
42
43 // Update text
44 function update(uint _index, string memory _text) public {
45     Todo storage todo = todos[_index];
46     todo.text = _text;
47 }
48
49 // Update completed
50 function toggleCompleted(uint _index) public {
51     Todo storage todo = todos[_index];
52     todo.completed = !todo.completed;
53 }
54 //Ved V Joshi
55 // Remove a Todo
56 function remove(uint _index) public {
57     delete todos[_index];
58 }
59 }
```

Explain contract

0 ☐ Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 15

LEARNETH 2.5.7 Learneth Solidity Beginner Course

8.4 Data Structures - Enums 15 / 19

8.4 Data Structures - Enums

In Solidity `enums` are custom data types consisting of a limited set of constant values. We use enums when our variables should only get assigned a value from a predefined set of values.

In this contract, the state variable `status` can get assigned a value from the limited set of provided values of the enum `Status` representing the various states of a shipping status.

Defining enums

We define an enum with the `enum` keyword, followed by the name of the custom type we want to create (line 6). Inside the curly braces, we define all available members of the enum.

Initializing an enum variable

We can initialize a new variable of an enum type by providing the name of the enum, the visibility, and the name of the variable (line 16). Upon its initialization, the variable will be assigned the value of the first member of the enum, in this case, Pending (line 7).

Even though enum members are named when you define them, they are stored as unsigned integers, not strings. They are numbered in the order that they were defined, the first member starting at 0. The initial value of `status`, in this case, is 0.

Compile Create a DApp with your contracts Start now

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Define enum with three possible sizes
6     enum Size {
7         S,
8         M,
9         L
10    }
11
12    // Initialize enum variable
13    //ved joshi
14
15    Size public sizes;
16
17    // Getter function
18    function getSize() public view returns (Size) {
19        return sizes;
20    }
21 }
22
```

Explain contract

0 ☐ Listen on all transactions

• ethers.js

Type the library name to see available commands.

Tut 16

Remix is moving to app.remix.live. Your Workspaces are stored in the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

LEARNETH 2.5.7 Learneth Solidity Beginner Course

9. Data Locations 16 / 19

the `function f` (line 12) and assign it the value of `myStruct`, changes in `myStructs` would not affect the values stored in the mapping `myStructs` (line 10). As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myStruct2` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myStruct2` member `foo` to 1.
3. Create a new struct `myStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myStruct2`, and `myStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer Show answer

Well done! No errors. Next

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7
8     struct MyStruct {
9         uint foo;
10    }
11
12    mapping(uint => MyStruct) myStructs;
13
14    function f() { infinite gas
15        public
16        returns (
17            MyStruct memory,
18            MyStruct memory,
19            MyStruct memory
20        )
21    }
22    // call _f with state variables
```

Tut 17

Remix is moving to app.remix.live. Your Workspaces are stored in the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

LEARNETH 2.5.7 Learneth Solidity Beginner Course

10.1 Transactions - Ether and Wei 17 / 19

Wei is the smallest subunit of *Ether*, named after the cryptographer *Wei Dai*. *Ether* numbers without a suffix are treated as `wei` (line 7).

`oneWei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`oneEther`

One `ether` is equal to 1,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer Show answer

Well done! No errors. Next

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6
7     // 1 wei is equal to 1
8     bool public isOneWei = 1 wei == 1;
9
10    // 1 gwei is equal to 10^9 wei
11    uint public oneGwei = 1 gwei;
12
13    // Check if 1 gwei is equal to 10^9 wei Ved Joshi
14    bool public isOneGwei = 1 gwei == 1e9;
15
16    uint public oneEther = 1 ether;
17
18    // 1 ether is equal to 10^18 wei
19    bool public isOneEther = 1 ether == 1e18;
20 }
```

Tut 18

Remix is moving to app.remix.live. Your Workspaces are stored in the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

LEARNETH 2.5.7 Learneth Solidity Beginner Course

10.2 Transactions - Gas and Gas Price 18 / 19

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on [ethereum.org](#).

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

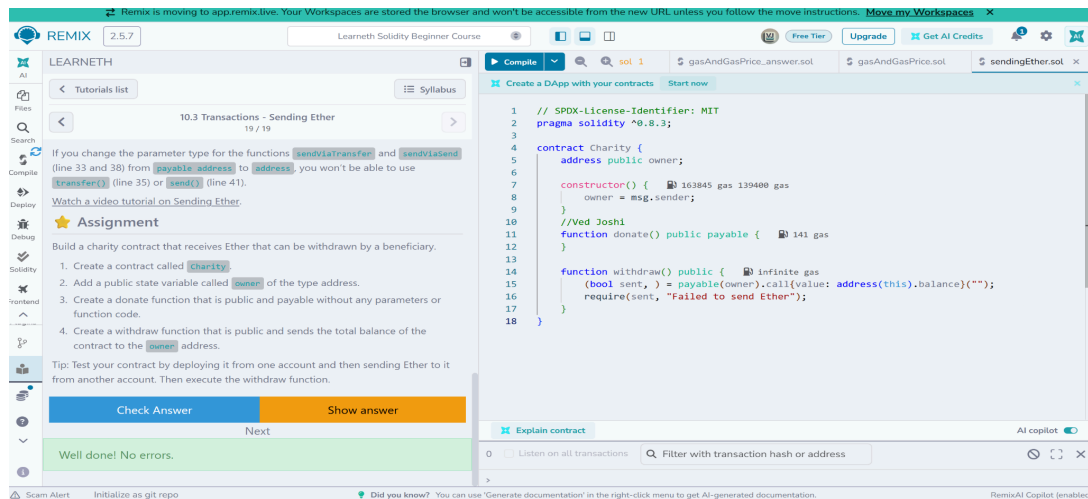
Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer Show answer

Well done! No errors. Next

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public { infinite gas
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        //Ved Joshi
15        while (true) {
16            i += 1;
17        }
18    }
19 }
```

Tut 19



Conclusion:

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.